

The Basics of Lua

The Basics of Lua



This tutorial can be completed without using the game. However it is recommended to use it as part of your workspace as this allows you to get used to SE

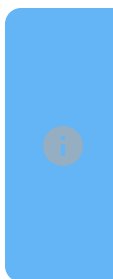


Don't know why you might want to look at this page? Check out [Getting started with Script Extender](#)

For modding Baldur's Gate we use the programming language [Lua](#)

The following section is aimed towards people who have no programming knowledge.

If you don't know what a function is, you might want to read through this. If you have worked with other programming languages before, feel free to skip this section but make sure to read up on Lua syntax. Its syntax is very similar to [python](#)



It is recommended to not only read this section but to follow along. Also try typing some of these commands instead of using copy and paste to better familiarize yourself with them

Use the example mod "MySEMod" to follow along by typing in `MyFirstSEScript.lua` from [Getting started with Script Extender](#) and resetting the console to see the changes.

1. Order of execution

Any script that you write will be executed from top to bottom.

```
1 | print("Hello World")
2 | print("My name is YourName")
```

Hello World

My name is YourName

This means that anything you define and want to refer to has to be declared before you will use it

```
1 | local myName = "YourName"
2 | print("My name is ", myName)
```

My name is YourName

When you use a variable before you have defined it, the program won't know what to do with it

```
1 | print("My name is ", myName)
2 | local myName = "YourName"
```

My name is nil



nil in Lua means nothing

2. Comments

You can also add lines to your script that don't do anything. These are **comments**

You can type them by adding two consecutive dashes `--` before your text.

```
1 | -- myVariable is 5
2 | local myVariable = 5
3 |
4 | -- prints myVariable
5 | print(myVariable)
```

5

You can also use comments to **comment out** lines you temporarily want to disable

```
1 | -- myVariable is 5
2 | local myVariable = 5
3 |
4 |
5 | -- prints myVariable
6 | -- print(myVariable)
```

(empty output)

3. Variables

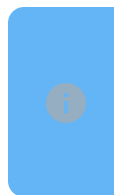
You can use variables to refer to certain values.

This has many applications. For example in BG3 we often use unique IDs (UUIDs) to refer to characters, objects, items and more.

For you to not have to retype these same long strings and maybe make mistakes, we can use variables

```
1 | local astarion = "S_Player_Astarion_c7c13742-
2 | bacd-460a-8f65-f864fe41f255"
   | print("Astarions ID is ", astarion)
```

Astarions ID is S_Player_Astarion_c7c13742-bacd-460a-8f65-f864fe41f255



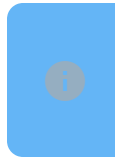
By default a value in Lua is global if you do not use the *local* keyword

In most cases you want your variables to be *local*

4. A very special data structure: tables

You might have heard of some data structures like **arrays** or **lists** before.

In Lua we use a so called **table** for all of these instances.

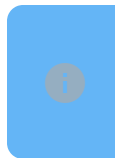


Paste the following function at the top of your `MyFirstSEScript.lua` so you can use it

```
1 | local function printMyTable(table)
2 |     for x,y in pairs(table) do
3 |         print(x, " ", y)
4 |     end
5 | end
```

Example usage:

```
1 | printMyTable(myTable)
```



To see the changes in your tables, use the `printMyTable` function after every change!

4.1. The basics

You can create a new table by instantiating an empty one

```
1 | local myTable = {}
```

(empty output)

You can also pre fill the table

```
1 | local myTable = {"Cheese", "Bread", "Wine"}
```

```
1 Cheese
2 Bread
3 Wine
```

You can add items to a table

```
1 | table.insert(myTable, "Grapes")
```

```
1 Cheese
2 Bread
3 Wine
4 Grapes
```

You can delete the last item from a table

```
1 | table.remove(myTable)
```

```
1 Cheese
2 Bread
3 Wine
```

You can delete an item from a table based on its index (position)

```
1 | table.remove(myTable, 1)
```

```
1 Bread
2 Wine
```

4.2. Iterating over a table

To be able to access all components, we want to iterate over the tables.

This is what we are doing in the `printMyTable` function.

```
1 | for x,y in pairs(myTable) do
2 |     print(x, " ", y)
3 | end
```

```
1 Bread
2 Wine
```

4.3. Different types of tables

4.3.1. Arrays

The tables in the example above can fit the description of an “array”.

They consist of an index and an entry

```
1 | local myTable = {"Cheese", "Bread", "Wine"}
```

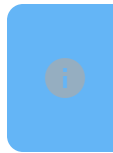
1 Cheese

2 Bread

3 Wine

```
1 | print("The first entry in myTable is ",
    myTable[1])
```

The first entry in myTable is Cheese



In most programming languages we start counting at 0, but in Lua we start at 1

4.3.2. Maps

We can also create “map-like” tables. These consist of “key - value” pairs. The **key** in this case is the entry on the left. The **value** is the entry on the right next to it. When we know the **key** we can immediately retrieve the **value**.

```
1 | ORIGINS = {
2 |     ["Wyll"] = "S_Player_Wyll_c774d764-4a17-
3 | 48dc-b470-32ace9ce447d",
4 |     ["ShadowHeart"] =
5 | "S_Player_ShadowHeart_3ed74f06-3c60-42dc-83f6-
6 | f034cb47c679",
7 |     ["Laezel"] = "S_Player_Laezel_58a69333-
8 | 40bf-8358-1d17-fff240d7fb12",
    ["Astarion"] = "S_Player_Astarion_c7c13742-
    bacd-460a-8f65-f864fe41f255",
    ["Gale"] = "S_Player_Gale_ad9af97d-75da-
    406a-ae13-7071c563f604",
    ["Karlach"] = "S_Player_Karlach_2c76687d-
    93a2-477b-8b18-8a14b549304c",
    }
```

Wyll S_Player_Wyll_c774d764-4a17-48dc-b470-32ace9ce447d

Karlach S_Player_Karlach_2c76687d-93a2-477b-8b18-8a14b549304c

Astarion S_Player_Astarion_c7c13742-bacd-460a-

8f65-f864fe41f255

Laezel S_Player_Laezel_58a69333-40bf-8358-1d17-fff240d7fb12

ShadowHeart S_Player_ShadowHeart_3ed74f06-3c60-42dc-83f6-f034cb47c679

Gale S_Player_Gale_ad9af97d-75da-406a-ae13-7071c563f604

```
1 | print("Wylls UUID is ", ORIGINS["Wyll"])
```

Wylls UUID is S_Player_Wyll_c774d764-4a17-48dc-b470-32ace9ce447d

4.3.3. Sets

The last tabel structure we will mention is a “set-like” table.

Sets are used to store unique items. That means duplicates are not allowed.

Here we can quickly look up if something exists in our set. For that, we set the entries to `true` .

```
1 | local creatures = {
2 |     ["Goblin"] = true,
3 |     ["Skeleton"] = true,
4 |     ["Elf"] = true
5 | }
```

Elf true

Goblin true

Skeleton true

```
1 | print("Is Elf part of creatures? ",
2 | creatures["Elf"])
   print("Is Dragonborn part of creatures? ",
       creatures["Dragonborn"])
```

Is Elf part of creatures? true

Is Dragonborn part of creatures? nil



In a lot of cases *nil* will give you the same result as *false*

4.4. Tables within tables

Tables are very flexible and allow varied structuring. Below you can see two more examples.

Note that because of the nested structure, our `printMyTable` function does not work anymore.

Instead we can use `_D` to “dump” the whole table and write special loops for them.

Example 1

```

1  local spells = {
2      ["fire"] = {
3          "Fireball",
4          "Flame Strike",
5          "Wall of Fire"
6      },
7      ["ice"] = {
8          "Ice Storm",
9          "Cone of Cold",
10         "Freeze"
11     }
12 }
13
14 _D(spells)

```

```

{
  "fire" :
  [
    "Fireball",
    "Flame Strike",
    "Wall of Fire"
  ],
  "ice" :
  [
    "Ice Storm",
    "Cone of Cold",
    "Freeze"
  ]
}

```



```

1 | for element, spellList in pairs(spells) do
2 |     print(element, " spells:")
3 |     for _, spell in ipairs(spellList) do
4 |         print(spell)
5 |     end
6 | end

```

ice spells:

Ice Storm

Cone of Cold

Freeze

fire spells:

Fireball

Flame Strike

Wall of Fire

Example 2

```

1 | local companion1 = {name = "Astarion", class =
2 | "Rogue"}
3 | local companion2 = {name = "Gale", class =
4 | "Wizard"}
5 | local companion3 = {name = "Karlach", class =
6 | "Barbarian"}
7 |
   | local myParty = {companion1, companion2,
   | companion3}
   |
   | _D(myParty)

```

output

```

[
  {
    "class" : "Rogue",
    "name" : "Astarion"
  },
  {
    "class" : "Wizard",
    "name" : "Gale"
  }
]

```


In this case `astarionUUID` is not known outside of its scope. If we want to access `astarionUUID` directly we can instantiate it outside of the scope of the function

```

1 | local astarionUUID =
2 | "S_Player_Astarion_c7c13742-bacd-460a-8f65-
   f864fe41f255"
   print(astarionUUID)

```

`S_Player_Astarion_c7c13742-bacd-460a-8f65-f864fe41f255`

This also means that when a variable is instantiated within one function, it is not known in any other functions

```

1 | -- returns Shadowhearts UUID
2 | local function getShart()
3 |     local shartUUID =
4 |     "S_Player_ShadowHeart_3ed74f06-3c60-42dc-83f6-
5 |     f034cb47c679"
6 |     return shartUUID
7 | end
8 |
9 | -- prints Shadowhearts UUID
10 | local function printShart()
11 |     print(shartUUID)
12 | end

    printShart()

```

`nil`

It is accessible again when we declare it outside of the scope of the function

```

1 | local shartUUID =
2 | "S_Player_ShadowHeart_3ed74f06-3c60-42dc-83f6-
3 | f034cb47c679"
4 |
5 | -- prints Shadowhearts UUID
6 | local function printShart()
7 |     print(shartUUID)

```

```
8 | end
```

```
printShart()
```

`S_Player_ShadowHeart_3ed74f06-3c60-42dc-83f6-f034cb47c679`

This also means that we can reuse names as long as they exist in different scopes

```
1 | -- returns Shadowhearts UUID
2 | local function getShart()
3 |     local uuid =
4 |     "S_Player_ShadowHeart_3ed74f06-3c60-42dc-83f6-
5 |     f034cb47c679"
6 |     return uuid
7 | end
8 |
9 | -- returns Astarions UUID
10 | local function getAstarion()
11 |     local uuid = "S_Player_Astarion_c7c13742-
12 |     bacd-460a-8f65-f864fe41f255"
13 |     return uuid
14 | end
15 |
```

```
print(getShart())
print(getAstarion())
```

`S_Player_ShadowHeart_3ed74f06-3c60-42dc-83f6-f034cb47c67`

`S_Player_Astarion_c7c13742-bacd-460a-8f65-f864fe41f255`

If one variable is defined in the outer scope it will get overwritten if it is being reused.

```
1 | local karlach = "S_Player_Karlach_2c76687d-
2 | 93a2-477b-8b18-8a14b549304c"
3 |
4 | local function switchKarlachWithLaezel()
5 |     karlach = "S_Player_Laezel_58a69333-40bf-
6 | 8358-1d17-fff240d7fb12"
7 |
```

```

/ |
8 | end
9 |
  | switchKarlachWithLaezel()
  |
  | print(karlach)

```

`S_Player_Laezel_58a69333-40bf-8358-1d17-fff240d7fb12`

5.2 Scope across files

if you have followed [5. Creating multiple files](#) from [Getting started with Script Extender](#) then you have two files in your “Server”.

`MyFirsSEScript.lua` and `MySecondSEScript.lua`

Not all variables and functions that you create in one file will be visible in the other one.

Create a variable in `MyFirsSEScript.lua`

```

1 | -- MyFirstSEScript.lua
2 |
3 | local gale = "S_Player_Gale_ad9af97d-75da-406a-ae13-7071c563f604"

```

Try to access it in `MySecondSEScript.lua`

```

1 | -- MySecondSEScript.lua
2 |
3 | print(gale)

```

`nil`

Because of the `local` keyword, the variable is not accessible in other files.

For that we have to make it a global variable by omitting the `local` keyword.

Create a variable in `MyFirsSEScript.lua`

```

1 | -- MyFirstSEScript.lua
2 |

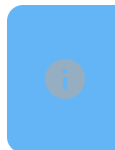
```

```
3 | Gale = "S_Player_Gale_ad9af97d-75da-406a-ae13-7071c563f604"
```

Try to access it in [MySecondSEScript.lua](#)

```
1 | -- MySecondSEScript.lua
2 |
3 | print(Gale)
```

[S_Player_Gale_ad9af97d-75da-406a-ae13-7071c563f604](#)



Global variables and functions usually start with an uppercase letter to distinguish them from local ones

The same rules apply to functions

Create a function in [MyFirstSEScript.lua](#)

```
1 | -- MyFirstSEScript.lua
2 |
3 | local function getWyll()
4 |     return "S_Player_Wyll_c774d764-4a17-48dc-b470-32ace9ce447d"
5 | end
```

Try to access it in [MySecondSEScript.lua](#)

```
1 | -- MySecondSEScript.lua
2 |
3 | print(getWyll())
```

`bg3se::lua::State::LoadScript(): Failed to execute script: [string "MySEMod/Server/MySecondSEScript.lua"]:1: attempt to call a nil value (global 'getWyll')`

Here you got your first intended error message. This is because we try to call the function `getWyll()` but `MySecondSeScript.lua` doesn't know how to access it.

Again we have to make the function global as well to access it from other files

Create a function in `MyFirstSEScript.lua`

```
1 | -- MyFirstSEScript.lua
2 |
3 | function GetWyll()
4 |     return "S_Player_Wyll_c774d764-4a17-48dc-
5 | b470-32ace9ce447d"
   | end
```

Try to access it in `MySecondSEScript.lua`

```
1 | -- MySecondSEScript.lua
2 |
3 | print(GetWyll())
```

`S_Player_Wyll_c774d764-4a17-48dc-b470-32ace9ce447d`



It is best practice to keep the content in your files separate most of the time.

Some content that will never change, like the ORIGIN UUIDs you might want to keep global in a separate *Data* folder

6. Functions

You can use functions when you have to execute the same code multiple times.

This allows you to reuse your code without having to copy and paste it. You will be less likely to make mistakes and your code will be easier to maintain

A function can be declared by using the keyword `function` . Then its `name` and parenthesis `()` .

It always has to end with `end`

```

1 | local function printAstarion()
2 |     print("S_Player_Astarion_c7c13742-bacd-
3 | 460a-8f65-f864fe41f255")
   | end

```

Optionally you can also add parameters to your function to make it more flexible.

Parameters allow you to add an input and reuse this input within a function.

In this case we name our parameter `companion` . You can use any name you like as long as it stays consistent within the function.

```

1 | local function printKarlachOrLaezel(companion)
2 |
3 |     if companion == "Karlach" then
4 |         print("S_Player_Karlach_2c76687d-
5 | 93a2-477b-8b18-8a14b549304c")
6 |
7 |     elseif companion == "Laezel" then
8 |         print("S_Player_Laezel_58a69333-40bf-
9 | 8358-1d17-fff240d7fb12")
10 |
11 |     else
12 |         print("Not Karlach or Laezel")
   |     end
   | end

```

You might want to add some descriptions to your functions so you and other people know what it does, what input it takes and what it returns.

```

1 | -- prints Astarions uuid
2 | local function printAstarion()
3 |     print("S_Player_Astarion_c7c13742-bacd-
4 | 460a-8f65-f864fe41f255")
5 | end
6 |
7 | -- prints Karlachs or Laezels uuid. Else prints

```



```

8 | an error message
9 | ---@param name string
10 | local function printKarlachOrLaezel(companion)
11 |
12 |     if companion == "Karlach" then
13 |         print("S_Player_Karlach_2c76687d-
14 | 93a2-477b-8b18-8a14b549304c")
15 |
16 |     elseif companion == "Laezel" then
17 |         print("S_Player_Laezel_58a69333-40bf-
18 | 8358-1d17-fff240d7fb12")
19 |
20 |     else
21 |         print("Not Karlach or Laezel")
22 |     end
23 | end

```

To call a function, simply type its name with the parenthesis.

```

1 | printAstarion()

```

[S_Player_Astarion_c7c13742-bacd-460a-8f65-f864fe41f255](#)

If you have added parameters for your function, then make sure to also include those.

```

1 | printKarlachOrLaezel("Karlach")
2 | printKarlachOrLaezel("Laezel")
3 | printKarlachOrLaezel("Gale")

```

[S_Player_Karlach_2c76687d-93a2-477b-8b18-8a14b549304c](#)

[S_Player_Laezel_58a69333-40bf-8358-1d17-fff240d7fb12](#)

[Not Karlach or Laezel](#)

Functions can also return values. You can save these return values in a variable to use later

```

1 | -- returns Astarions uuid
2 | ---@return uuid string
3 | local function getAstarion()
4 |     return "S_Player_Astarion_c7c13742-bacd-
5 | 460a-8f65-f864fe41f255"
   | end

```

```

1 | local astarion = getAstarion()
2 |
3 | print(astarion)

```

[S_Player_Astarion_c7c13742-bacd-460a-8f65-f864fe41f255](#)

This also works for functions that take parameters

```

1 | -- returns Karlachs or Laezels uuid. Else
2 | returns nil
3 | ---@param name string
4 | local function getKarlachOrLaezel(companion)
5 |
6 |     if companion == "Karlach" then
7 |         return "S_Player_Karlach_2c76687d-93a2-
8 | 477b-8b18-8a14b549304c"
9 |
10 |    elseif companion == "Laezel" then
11 |        return "S_Player_Laezel_58a69333-40bf-
12 | 8358-1d17-fff240d7fb12"
   |
   |     end
   | end

```

```

1 | local karlach = getKarlachOrLaezel("Karlach")
2 | local laezel = getKarlachOrLaezel("Laezel")
3 | local gale = getKarlachOrLaezel("Gale")
4 |
5 | print(karlach)
6 | print(laezel)
7 | print(gale)

```

```
S_Player_Karlach_2c76687d-93a2-477b-8b18-8a14b549304c  
S_Player_Laezel_58a69333-40bf-8358-1d17-fff240d7fb12  
nil
```

7. If statements

For now we have written code that always executes the same way. To be able to make it more interactive we can use **flow statements** to change the way our code is executed. First we will have a look at if statements.

The general structure of an if statement is

```
1 | if condition then  
2 |     -- Your code  
3 | end
```

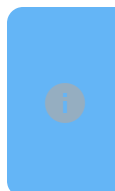
`condition` has to evaluate to `true` for the code underneath the if statement to be executed.

Here are some examples for conditions that evaluate to `true`

```
1 | if 1 == 1 then  
2 |     print("1 is indeed equal to 1")  
3 | end  
4 |  
5 | if "Astarion" then  
6 |     print("Astarion exists")  
7 | end
```

1 is indeed equal to 1

Astarion exists



In Lua, when a value exists, it will automatically evaluate to `true`.

That is why "Astarion" evaluates to `true`

If `condition` evaluates to `false` then the code underneath the `if` statement is not executed.

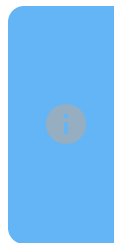
Here are some examples for conditions that evaluate to `false`

```

1 | if 0 == 1 then
2 |     print("0 is indeed equal to 1")
3 | end
4 |
5 | if nil then
6 |     print("Astarion exists")
7 | end

```

(empty output)



In Lua, when a value does not exist, it will automatically evaluate to `false`.

Since `nil` means nothing, our condition evaluates to `false`

To be able to use your `if` statement flexibly, you do not want to predefine the condition

```

1 | local myNumber = 5
2 |
3 | if myNumber == 5 then
4 |     print("My number is 5")
5 | end

```

My number is 5

You can have your `if` statement in a function to make it even more flexible

```

1 | local myFour = 4
2 | local myFive = 5
3 |
4 | -- prints "Yes" if the number is five, else
5 | prints "No"
6 | ---@param number int
7 | local function isItFive(number)

```

```
8 |     if number == 5 then
9 |         print("Yes")
10 |     else
11 |         print("No")
12 |     end
13 | end
14 |
15 | isItFive(myFour)
    isItFive(myFive)
```

No

Yes

You can also check for something to **not** be true by using the **~=** operator

```
1 | local myNumber = 0
2 |
3 | if myNumber ~= 5 then
4 |     print("My number is not 5")
5 | end
```

My number is not 5

You can also use multiple conditions in your statement to check if either one **or** another condition is true

```
1 | local x = 0
2 | local y = 1
3 |
4 | if x == 1 or y == 1 then
5 |     print("Either x or y are 1")
6 | end
```

Either x or y are 1

You can also use multiple conditions in your statement to check if one **and** another condition are true

```
1 | local x = 1
2 | local y = 1
3 |
```

```

3
4
5  if x == 1 and y == 1 then
6      print("Both x and y are 1")
    end

```

Both x and y are 1

You can stack as many of these statements as you like.



Beware of a common pitfall when evaluating conditions!
 Since the existence of a value is enough to have it evaluate to true
 never forget to actually compare it to the desired value.

```

1  local x = 0
2  local y = 0
3
4  if 1 == x or y then
5      print("Either x or y are 1")
6  end

```

Either x or y are 1 (wrong output!)

Even though when reading the statement `if 1 == x or y then` out loud, it might seem like we are comparing both `x` and `y` to `1`, we are not. We are comparing `x` to `1` and checking if `y` simply exists.

Your variables always have to be compared to the desired value

```

1  local x = 0
2  local y = 0
3
4  if 1 == x or 1 == y then
5      print("Either x or y are 1")
6  end

```

(empty output - correct!)

You are not simply restricted to one case with if statements. Using `elseif` you can check if multiple things are true

```
1 | local x = 5
2 |
3 | if x == 1 then
4 |     print("x is 1")
5 | elseif x == 2 then
6 |     print("x is 2")
7 | elseif x == 3 then
8 |     print("x is 3")
9 | elseif x == 4 then
10 |    print("x is 4")
11 | elseif x == 5 then
12 |    print("x is 5")
13 | else
14 |     print("x is not between 1 and 5")
15 | end
```

x is 5

8. Loops

Another way to make our code more interactive is by the use of **loops** to repeat certain steps.

You have already used in if you have followed **3. A very special data structure: tables** where we used the function `printMyTable` to iterate over a table.

There are different types of loops. We will only cover the `for loop` here.

They will execute their body for a set amount of iterations.

```
1 | -- prints the count from 1 to 5
2 | for i = 1,5 do
3 |     print(i)
4 | end
```

1
2
3
4
5

We usually do not want to iterate for a set amount of loops, but base it on a table.

using `#tableName` you can get the size of a table.

```
1 | local myTable = {"Cheese", "Bread", "Wine"}
2 |
3 | for i=1,#myTable do
4 |     print(myTable[i])
5 | end
```

Cheese

Bread

Wine

You can also iterate over the table without directly accessing its indices

```
1 | local myTable = {"Cheese", "Bread", "Wine"}
2 |
3 | for _,food in pairs(myTable) do
4 |     print(food)
5 | end
```

Cheese

Bread

Wine

Of course you can also use this in a function

```
1 | local myTable = {"Cheese", "Bread", "Wine"}
2 |
3 |
4 | -- prints the content from an array like table
5 | local function printArraylike(table)
6 |     for _,content in pairs(table) do
7 |         print(content)
8 |     end
9 | end
10 |
11 | printArraylike(myTable)
```


Cheese

Bread

Wine

As you have seen in **3.4. Tables within tables** we can *nest* tables. That means we can put a table within another table.

To retrieve the values in these **nested tables** we can use **nested loops**

```
1  local cheeseTable = {"Gorgonzola", "Cheddar",  
2  "Brie"}  
3  local breadTable = {"Black Bread", "Whole Grain  
4  Bread", "Rye Bread"}  
5  local wineTable = {"Merlot", "Chardonnay",  
6  "Cabernet Sauvignon"}  
7  
8  local foodTable = {myCheeseTable,  
9  myBreadTable, myWineTable}  
10  
11  
12  -- prints the content from a table that  
13  contains tables  
14  local function printNestedTable(nestedTable)  
15      for _,table in pairs(nestedTable) do  
16          for _,content in pairs(table) do  
              print(content)  
          end  
      end  
  end  
  
  printNestedTable(foodTable)
```

Gorgonzola

Cheddar

Brie

Black Bread

Whole Grain Bread

Rye Bread

Merlot

Chardonnay

Cabernet Sauvignon

9. Debugging

9.1 Debugging using error output

While following this guide you likely already ran into a few errors. These are shown in red on the console:

```

Error while dispatching event keyInput: [string "Scribe/Client/Scribe.lua"]:11: bad argument #3 to '___newindex' (string expected, got nil)
stack traceback:
  [C++ Code]: in metamethod '___newindex'
  Scribe/Client/Scribe.lua:11: in function <Scribe/Client/Scribe.lua:5>
  [C++ Code]: in Function 'xpcall'
  builtin://libs/Event.lua:129: in method 'Throw'
  builtin://libs/Event.lua:184: in function <builtin://libs/Event.lua:182>

```

For this we will write some code that will throw an error:

```

1 | local function myMistake(num1, num2)
2 |     print("num1" + num_2)
3 |
4 | myMiftake("Astarion")

```

```

bg3se::lua::State::LoadScript(): Failed to
parse script: [string
"MySEMod/MyFirstSEScript.lua"]:4: 'end'
expected (to close 'function' at line 1) near
<eof>

```

Here we get the error message **Failed to parse script**. This means that we already made a mistake with writing our script regardless of the input.

If like me you started writing at **line 1** you will find the mistake in the same line as well.

[string "MySEMod/MyFirstSEScript.lua"]:4: shows us that the issue is in **MyFirstSEScript.lua** in line **4**.

'end' expected (to close 'function' at line 1) shows us what the error is.

We forgot to close the function we start in line **1** with an **end**. So let's add this

```

1 | local function myMistake(num1, num2)
2 |     print("num1" + num_2)
3 | end
4 |
5 | myMiftake("Astarion")

```

```
bg3se::lua::State::LoadScript(): Failed to
execute script: [string
"Scribe/BootstrapServer.lua"]:5: attempt to
call a nil value (global 'myMiftake')
```

Now the issue is `Failed to execute script`. This means the code we wrote is at least valid but we made a mistake in executing it.

```
[string "Scribe/BootstrapServer.lua"]:5:
attempt to call a nil value (global
'myMiftake')
```

We have an issue in line 5 where we call a `nil value`, something that doesn't exist.

If you are observant you can spot that here we made a small typo.

Instead of calling `myMistake` we called `MyMiftake` which doesn't exist.

```
myMiftake("Astarion") -> myMistake("Astarion")
```

Let us fix this typo

```
1 | local function myMistake(num1, num2)
2 |     print("num1" + num_2)
3 | end
4 |
5 | myMistake("Astarion")
```

```
bg3se::lua::State::LoadScript(): Failed to
execute script: [string
"Scribe/BootstrapServer.lua"]:2: attempt to
perform arithmetic on a string value
```

The next error is `attempt to perform arithmetic on a string value` in line 2

We mistakenly used `"num1"` instead of `num1`, so we cannot perform additions.

```
print("num1" + num_2) -> print(num1 + num_2)
```

```
1 | local function myMistake(num1, num2)
2 |     print(num1 + num_2)
3 | end
4 |
```

```

5 | myMistake("Astarion")

```

bg3se::lua::State::LoadScript(): Failed to execute script: [string "Scribe/BootstrapServer.lua"]:2: attempt to perform arithmetic on a string value (local 'num1')

Here we have another attempt to perform arithmetic on a string value but this time we have some more additional information (local 'num1')

This means that the parameter we pass to this function num1 is a string but we cannot perform additions on strings.

So let us change that

myMistake("Astarion") -> myMistake(5)

```

1 | local function myMistake(num1, num2)
2 |     print(num1 + num_2)
3 | end
4 |
5 | myMistake(5)

```

bg3se::lua::State::LoadScript(): Failed to execute script: [string "Scribe/BootstrapServer.lua"]:2: attempt to perform arithmetic on a nil value (global 'num_2')

This time we get the error attempt to perform arithmetic on a nil value (global 'num_2')

This means num2 does not exist. And indeed we do not pass a second argument when calling the function

myMistake(5) -> myMistake(5, 4)

```

1 | local function myMistake(num1, num2)
2 |     print(num1 + num_2)
3 | end
4 |
5 | myMistake(5, 4)

```

```
myMistake(5, 4)
```

```
bg3se::lua::State::LoadScript(): Failed to
execute script: [string
"Scribe/BootstrapServer.lua"]:2: attempt to
perform arithmetic on a nil value (global
'num_2')
```

We still get the same error message as before. We know that we pass a number as `num2`, so we need to check the function.

It looks like we mistakenly used `num2` as the name of the parameter but later in the print statement we used `num_2`

```
print(num1 + num_2) -> print(num1 + num2)
```

```
1 | local function myMistake(num1, num2)
2 |     print(num1 + num2)
3 | end
4 |
5 | myMistake(5, 4)
```

9

Seems like we were able to successfully debug our code with the help of errors!

9.2 Debugging using print statements

However, mistakes aren't always this obvious. Sometimes our code is "valid"

but we still do not get the desired output.

We will go over how to use `print` statements to debug our code.

```
1 | local function isAstarion(uuid)
2 |     if uuid == "S_Player_Astarion_c7c13742-
3 | bacd-460a-8f65-f864fe41f255" then
4 |         return 1
5 |     else
6 |         return 0
7 |     end
8 | end
9 |
```

```

10 |
11 |
12 | if isAstarion("S_Player_Astarion_c7c13742-bacd-
    460a-8f65-f864fe41f255") == true then
        print("Astarion found")
    end

```

(empty output)

We expect `"Astarion found"` to be printed but instead we get nothing so we don't know where to start looking for mistakes.

Let's start by adding an `else` statement to our `if statement`. Right now we get no output, but we suspect that the `if statement` evaluates to false which leads to the `print` statement not being executed.

```

1 | local function isAstarion(uuid)
2 |     if uuid == "S_Player_Astarion_c7c13742-
3 | bacd-460a-8f65-f864fe41f255" then
4 |         return 1
5 |     else
6 |         return 0
7 |     end
8 | end
9 |
10 |
11 | if isAstarion("S_Player_Astarion_c7c13742-bacd-
12 | 460a-8f65-f864fe41f255") == true then
13 |     print("Astarion found")
14 | else
        print("Astarion not found")
    end

```

`Astarion not found`

Now we know for certain that the issue is in the evaluation of our `if statement`.

We expect it to evaluate to true but it does not.

Let's go further and gave a look at our function

`isAstarion(uuid)`

and add some `print` statements here as well.

```

1  local function isAstarion(uuid)
2      if uuid == "S_Player_Astarion_c7c13742-
3  bacd-460a-8f65-f864fe41f255" then
4          print(uuid , " is Astarion")
5          return 1
6      else
7          print(uuid , " is not Astarion")
8          return 0
9      end
10 end
11
12
13 if isAstarion("S_Player_Astarion_c7c13742-bacd-
14 460a-8f65-f864fe41f255") == true then
15     print("Astarion found")
16 else
17     print("Astarion not found")
18 end

```

S_Player_Astarion_c7c13742-bacd-460a-8f65-f864fe41f255 is Astarion
Astarion not found

Our function `isAstarion()` works and returns the correct value, but our `if statement` evaluates to false, so there must be a mistake in how we compare the result.

Let us add another statement so we can directly compare.

```

1  local function isAstarion(uuid)
2      if uuid == "S_Player_Astarion_c7c13742-
3  bacd-460a-8f65-f864fe41f255" then
4          print(uuid , " is Astarion")
5          print("returning 1")
6          return 1
7      else
8          print(uuid , " is not Astarion")
9          return 0
10     end
11 end
12
13
14 if isAstarion("S_Player_Astarion_c7c13742-bacd-
15

```

```

16 | 460a-8f65-f864fe41f255") == true then
17 |     print("Astarion found")
18 | else
    print("Astarion not found")

    print(isAstarion("S_Player_Astarion_c7c13742-
    bacd-460a-8f65-f864fe41f255"), " is not the
    same as ", true)
end

```

```

S_Player_Astarion_c7c13742-bacd-460a-8f65-
f864fe41f255 is Astarion
returning 1
Astarion not found
S_Player_Astarion_c7c13742-bacd-460a-8f65-
f864fe41f255 is Astarion
returning 1
1 is not the same as true

```

Here we can see our mistake. In `isAstarion(uuid)` we return `0` or `1` but in our `if statement` we check whether something is `true`.

We have to change our `if statement` to reflect the correct return values of the function `isAstarion(uuid)`

```

1 | local function isAstarion(uuid)
2 |     if uuid == "S_Player_Astarion_c7c13742-
3 | bacd-460a-8f65-f864fe41f255" then
4 |         print(uuid , " is Astarion")
5 |         print("returning 1")
6 |         return 1
7 |     else
8 |         print(uuid , " is not Astarion")
9 |         return 0
10 |     end
11 | end
12 |
13 |
14 | if isAstarion("S_Player_Astarion_c7c13742-bacd-
15 | 460a-8f65-f864fe41f255") == 1 then
16 |     print("Astarion found")
17 | else
18 |     print("Astarion not found")

```



```
print(isAstarion("S_Player_Astarion_c7c13742-
bacd-460a-8f65-f864fe41f255"), " is not the
same as ", true)
end
```

S_Player_Astarion_c7c13742-bacd-460a-8f65-f864fe41f255 is Astarion
 returning 1
 Astarion found

Now we get the correct output.

We can delete, or comment out our print statement so they don't clutter up our console.

```
1  local function isAstarion(uuid)
2      if uuid == "S_Player_Astarion_c7c13742-
3  bacd-460a-8f65-f864fe41f255" then
4          return 1
5      else
6          return 0
7      end
8  end
9
10
11  if isAstarion("S_Player_Astarion_c7c13742-bacd-
12  460a-8f65-f864fe41f255") == 1 then
    print("Astarion found")
end
```

Astarion found

10. A brief introduction to metatables



This section is about a slightly advanced topic. Feel free to skip it if it sounds too complicated.

It can be helpful to come back here when your projects grow in size since we discuss one concept of

Object-Oriented-Programming (OOP) - read more here:

https://en.wikipedia.org/wiki/Object-oriented_programming 

Lua is a very flexible language. Through its usage of tables multiple OOP concepts like Inheritance can be realized.
Here we will talk about *Metatables* which is the Lua equivalent of *Objects*.

For this section we will rename `MyFirstSEScript.lua` to `Main.lua` and `MySecondSEScript.lua` to `Companion.lua`, to better showcase this principle.

Note: Make sure to also modify the script names in `BootstrapServer.lua`.

`Companion.lua` must be loaded before `Main.lua` since we use components of `Companion.lua` in `Main.lua`.

```
1 | Ext.Require("Server/Companion.lua")
2 | Ext.Require("Server/Main.lua")
```

in `Companion.lua` we will now build a `Companion`
First we have to create our metatable

```
1 | -- Companion.lua
2 |
3 | -- creating the metatable
4 |
5 | Companion = {}
6 | Companion.__index = Companion
```

Now we have to decide what kind of components we need to give a companion when we create them.

We are simply going to choose *name*, *health* and *class*

```
1 | ---@param name string
2 | ---@param health int
3 | ---@param class string
4 | function Companion:new(name, health, class)
5 |     local instance = setmetatable({
6 |         name = name,
7 |
```

```

7         health = health,
8         class = class
9     }, Companion)
10     return instance
11 end

```

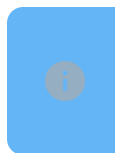
If there are any additional components of a [Companion](#) we can add them now.

Here I chose to add all classes in BG3 as a set like table.

```

1  -- constants (you could also make this global
2  since you should never change this)
3
4  local allowedClasses = {
5      Barbarian = true,
6      Bard = true,
7      Cleric = true,
8      Druid = true,
9      Fighter = true,
10     Monk = true,
11     Paladin = true,
12     Ranger = true,
13     Rogue = true,
14     Sorcerer = true,
15     Warlock = true,
16     Wizard = true
    }

```



For the simplicity of this example we do not check for allowed classes when creating a companion

We also want to access the components of our [Companion](#)

```

1  -- getters methods
2
3  -- returns the companions name
4  ---@return name string
5  function Companion:getName()
6      return self.name
7  end
8
9
10

```

```

10  -- returns the companions health
11  ---@return health int
12  function Companion:getHealth()
13      return self.health
14  end
15
16  -- returns the companions class
17  ---@return class string
18  function Companion:getClass()
19      return self.class
20  end

```

And we want to modify our companions

```

1  -- methods for modifying companions
2
3  -- increases the companions health
4  ---@param healAmount int    - the amount by
5  what the companion should be healed
6  function Companion:heal(healAmount)
7      -- We can only heal someone is the
8  healAmount is larger than 0
9      if healAmount > 0 then
10         self.health = self.health + healAmount
11         print(self.name , " has been healed for
12  ", healAmount)
13         print("their health has increased to ",
14  self.health)
15     else
16         print("Negative healAmounts are not
17  allowed")
18     end
19 end
20
21
22 -- changes the companions class
23 ---@param class string - the new class of the
24 companion
25 function Companion:respec(newClass)
26     -- check if the class is in allowedClasses
27     if allowedClasses[newClass] then
28         self.class = newClass
29         print(self.name, " has changed their
30  class to ", self.class)
31     else
32         print(newClass, " is not an allowed

```

```
class")  
    end  
end
```

Now that our `Companion` class is done, we can use it in `Main.lua`. Below you can see a few examples but feel free to experiment.

Creating the companions:

```
1 | -- Main.lua  
2 |  
3 | -- Creating a companion named Astarion  
4 | local astarion = Companion:new("Astarion", 100,  
5 | "Rogue")  
6 |  
7 | -- Creating another companion named Gale  
   local gale = Companion:new("Gale", 120,  
   "Wizard")
```

Retrieving their components:

```
1 | -- Retrieving the companions' components  
2 | print("Astarion - Name:", astarion:getName(),  
3 | "Health:", astarion:getHealth(), "Class:",  
4 | astarion:getClass())  
  
   print("Gale - Name:", gale:getName(),  
   "Health:", gale:getHealth(), "Class:",  
   gale:getClass())
```

Astarion - Name: Astarion Health: 100 Class:
Rogue

Gale - Name: Gale Health: 120 Class: Wizard

Healing the companions:

```
1 | -- Healing a companion  
2 | print("Before healing, Astarion's health:",  
3 | astarion:getHealth())  
4 | astarion:heal(20)  
5 |
```

```

6 | print("After healing, Astarion's health:",
7 |   astarion:getHealth())
8 |
9 | -- Trying to heal a companion with negative
   health
   print("Before healing, Gale's health:",
   gale:getHealth())
   gale:heal(-50)
   print("After attempting negative healing,
   Gale's health:", gale:getHealth())

```

Before healing, Astarion's health: 100
 Astarion has been healed for 20
 their health has increased to 120
 After healing, Astarion's health: 120

Before healing, Gale's health: 120
 Negative healAmounts are not allowed
 After attempting negative healing, Gale's
 health: 120

```

1 | -- Respeccing a companion to a legal class
2 | print("Astarion's class before respec:",
3 |   astarion:getClass())
4 |   astarion:respec("Paladin")
5 |   print("Astarion's class after respec:",
6 |     astarion:getClass())
7 |
8 | -- Respeccing a companion to an illegal class
9 | print("Gale's class before trying illegal
   respec:", gale:getClass())
   gale:respec("Necromancer")
   print("Gale's class after trying illegal
   respec:", gale:getClass())

```

Astarion's class before respec: Rogue
 Astarion has changed their class to Paladin
 Astarion's class after respec: Paladin

Gale's class before trying illegal respec:
 Wizard
 Necromancer is not an allowed class

Gale's class after trying illegal respec: Wizard



Want to practice more Lua?

Visit exercism: <https://exercism.org/tracks/lua> 

Powered by [Wiki.js](#)